ensemble-next

Claude Code setzt um, Codex prüft blind.

Dokumentation zum Beta-Stand

Version	1.2.0-beta.1
Stand	13. November 2026
Repository	https://github.com/schrauberMK/ensemble-next
Lizenz	MIT, ohne Gewährleistung
Ursprung	Abkömmling von https://github.com/michelhelsdingen/ensemble

Beta. ensemble-next ist ein Werkzeug aus dem eigenen Gebrauch und kein fertiges Produkt. Es kann Fehler enthalten, sich ohne Ankündigung ändern und verbraucht bei jedem Lauf Kontingent oder Geld bei den Anbietern der KI-Modelle. Die Agenten ändern Dateien ohne Rückfrage. Die Nutzung geschieht auf eigene Verantwortung. Es gibt keine Gewährleistung und keine Zusage, dass Fehler behoben oder Fragen beantwortet werden.

Inhaltsverzeichnis

1 Worum es geht **3**

2 Stand, Haftung und was du wissen solltest **3**

3 Herkunft und verwandte Projekte **3**

4 Die zwei Arbeitsweisen **4**
4.1 Standard: Claude Code allein, Codex prüft . . . 4
4.2 Team: Codex leitet, Claude Code setzt um . . . 4
4.3 Der Vergleich . . . 5
4.4 So wählst du . . . 5

5 Welche Modelle arbeiten und wie du sie einstellst **5**
5.1 Claude Code auf ein Modell festlegen . . . 6
5.2 Codex im Team festlegen . . . 6
5.3 Den Prüfer festlegen . . . 6
5.4 Was beim Einstellen zählt . . . 7

6 Die Sicherungen im Einzelnen **7**
6.1 Abschlusscheck . . . 7
6.2 Abbruchregeln für die Nacharbeit . . . 7
6.3 Blinde Prüfung . . . 8
6.4 Zitatprüfung . . . 8
6.5 Bindung an die Prüfsumme . . . 8
6.6 Abgeschotteter Prüfer . . . 8

7 Einrichtung und erster Lauf **9**
7.1 Voraussetzungen . . . 9
7.2 Einrichten . . . 9
7.3 Erster Lauf . . . 9
7.4 Ergebnis lesen . . . 9

8 Schalter **10**

9 Messwerte **10**

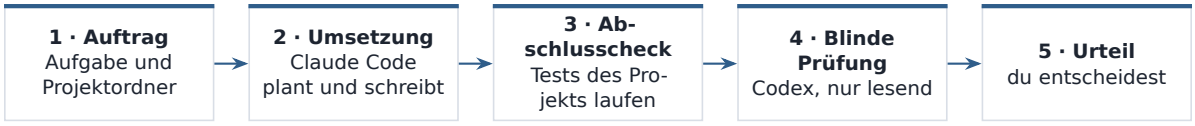
10 Grenzen und bekannte Punkte **11**

11 Lizenz und Haftung **11**

1 Worum es geht

Wer mit einem KI-Programmierwerkzeug arbeitet, kennt das Problem: Das Werkzeug meldet „fertig, alle Tests grün“, und ob das stimmt, zeigt sich erst beim Nachsehen. ensemble-next nimmt dieses Nachsehen zu einem guten Teil ab. Es lässt **Claude Code** eine Aufgabe umsetzen und danach **Codex** das Ergebnis prüfen. Codex kommt von einem anderen Hersteller, sieht den Arbeitsverlauf nicht und darf nichts ändern. Es beurteilt nur, was am Ende im Projektordner steht.

Dazwischen sitzen feste, nicht von einem Modell abhängige Kontrollen: Die Tests des Projekts laufen wirklich, das Urteil des Prüfers wird auf erfundene Fundstellen untersucht, und es wird festgehalten, zu welchem Stand das Urteil gehört.



Am Ende steht ein Urteil in einer Textdatei. Committet wird nichts von selbst. Das bleibt deine Entscheidung, nachdem du Urteil und Diff gesehen hast.

2 Stand, Haftung und was du wissen solltest

Beta. Der Standardablauf ist an kleinen Aufgaben gemessen und im täglichen Gebrauch. Die vier Ergänzungen vom November 2026 (Abschnitt 6) sind durch automatische Tests abgedeckt und an wenigen echten Läufen erprobt, aber noch nicht lange im Einsatz.

Getestete Umgebung. Ubuntu Linux, Node.js 26, tmux 3.6, Python 3, Claude Code 2.1.322, codex-cli 0.167.1. Beide Kommandozeilenprogramme werden häufig aktualisiert. Ein Update kann den Ablauf stören; das ist während der Entwicklung mehrfach passiert. macOS ist vom Ursprungsprojekt her vorgesehen, hier aber nicht getestet. Windows wird nicht unterstützt.

Rechte der Agenten. Die Agenten laufen ohne Rückfragen. Claude Code startet mit `--permission-mode auto`, Codex als Teammitglied mit `--dangerously-bypass-approvals-and-sandbox`. Sie können im Projektordner alles ändern und Befehle ausführen. Nimm einen Ordner, in dem das vertretbar ist, und arbeite mit Git.

Kosten. Jeder Lauf verbraucht Kontingent eines Abos oder Geld über die Schnittstelle. In der Messreihe lag ein Lauf mit Claude bei rund 0,40 US-Dollar, gerechnet zum Listenpreis der Schnittstelle. Die Prüfung durch Codex kostet etwa 10 000 Tokens.

Daten. Aufgabe und Code gehen an Anthropic und OpenAI, so wie bei der normalen Nutzung der beiden Programme. ensemble-next schickt von sich aus nichts an Dritte. Benachrichtigungen über Telegram oder einen eigenen Alarm-Dienst gibt es nur, wenn du sie selbst einrichtest.

Keine Gewährleistung. Die Software wird bereitgestellt, wie sie ist (Abschnitt 11).

3 Herkunft und verwandte Projekte

ensemble-next ist nicht aus dem Nichts entstanden. Diese Projekte gehören dazu oder haben es beeinflusst:

Projekt	Bezug
michelhelsdingen/ensemble MIT	**Ursprungsprojekt.** ensemble-next ist ein Abkömmling davon. Server, Nachrichtenbus, Monitor, die Team-Arbeitsweise und die meisten Skripte stammen von dort. Der Urheberrechtsvermerk bleibt in der Lizenzdatei erhalten.
nyldn/claude-octopus MIT	**Ideengeber.** Ein Plugin, das viele Modelle gegeneinander prüfen lässt. Von dort stammen die Anregungen für Zitatprüfung, Abbruchregeln, Bindung an die Prüfsumme und den abgeschotteten Prüfer. Es wurde kein Code übernommen. ensemble-next geht bewusst den kleineren Weg: ein Agent arbeitet, einer prüft.
cubetribe/ClaudeCode_GodMode-On proprietär	**CC_GodMode.** Ein Regelwerk für Claude Code mit Rollen, Qualitätsstufen und Berichten. Eigenständig und *nicht* in diesem Repository enthalten. ensemble-next braucht es nicht und verträgt sich damit. Der Abschlusscheck schließt eine Lücke, die dort offen bleibt: GodMode prüft beim Ende eines Unteragenten, ensemble-next auch dann, wenn Claude selbst schreibt.
cubetribe/CODEX_GodMode_ON proprietär	**Das Gegenstück für Codex.** Ebenfalls nicht enthalten. Wer es installiert hat, findet seine Regeln in `~/.codex/AGENTS.md`; diese Datei lädt Codex auch als Prüfer (Abschnitt 10).
Claude Code und Codex CLI	Die beiden Werkzeuge, die hier zusammenarbeiten. ensemble-next startet sie nur und steht in keiner Verbindung zu Anthropic oder OpenAI.

Den Anstoß zum Umbau gab eine Empfehlung von Anthropic, eng gekoppelte Planungs- und Ausführungsagenten zu einem Hauptagenten zusammenzulegen. Ob das für die eigenen Aufgaben stimmt, sollte eine Messung zeigen und nicht die Empfehlung allein (Abschnitt 9).

4 Die zwei Arbeitsweisen

ensemble-next kennt zwei Wege, eine Aufgabe zu erledigen. Du wählst beim Start, welcher gilt. Eine automatische Wahl gibt es nicht.

4.1 Standard: Claude Code allein, Codex prüft

Ohne weitere Angabe arbeitet Claude Code allein. Es liest das Projekt, plant, schreibt den Code und lässt die Tests laufen, alles in einem Zusammenhang. Meldet es am Ende „bereit zur Prüfung“, startet der Server die blinde Prüfung durch Codex.

Das ist der Normalfall für abgegrenzte Aufgaben: einen Fehler beheben, eine Funktion ergänzen, ein Modul umbauen.

4.2 Team: Codex leitet, Claude Code setzt um

Nennst du beim Start `codex,claude`, bilden beide ein Team. Der zuerst genannte Agent leitet: Codex liest die Aufgabe, plant und übergibt einen Arbeitsauftrag. Claude Code setzt ihn um und meldet zurück. Codex nimmt ab oder fragt nach. Beide tauschen sich über einen Nachrichtenkanal aus, dem du zusehen kannst. Danach läuft dieselbe blinde Prüfung wie im Standardfall, in einer frischen Codex-Sitzung ohne Kenntnis der Teamarbeit.

Das Team lohnt sich, wenn zwei Sichtweisen gefragt sind: bei offenen Entwurfsfragen, bei Recherche oder wenn ein zweites Modell den Plan mittragen soll. Weitere Agenten sind möglich, etwa ein dritter als Rechercheur und kritischer Fragesteller. Für typische Aufgaben gibt es Rollenvorlagen (`review`, `implement`, `research`, `debug`).

4.3 Der Vergleich

	Standard	Team
Aufruf	ohne Angabe von Agenten	`codex,claude` als drittes Argument oder `COLLAB_AGENTS=codex,claude`
Planung	Claude Code	Codex
Umsetzung	Claude Code	Claude Code
Abschlusscheck	ja	ja, für Claude Code
Blinde Prüfung danach	Codex, frische Sitzung	Codex, frische Sitzung
Ausstattung von Codex	Prüfer ohne MCP-Server, Plugins, Apps und Erinnerungen	Team-Codex mit deiner vollen Codex-Konfiguration; der Prüfer danach wieder abgeschottet
Rechte von Codex	nur lesen, leerer Ordner	schreiben im Projektordner
Dauer in der Messreihe	57 s (Median)	156 s (Median)

Warum Codex in den beiden Rollen verschieden ausgestattet ist. Der Prüfer soll nur ein Bündel lesen und urteilen. Alles Weitere kostet bei jeder Prüfung Tokens und macht das Urteil davon abhängig, was gerade installiert ist. Deshalb startet er ohne die persönliche Codex-Konfiguration. Der Team-Codex dagegen arbeitet mit: Er soll recherchieren, Werkzeuge nutzen und Dateien ändern können. Deshalb bekommt er alles, was deine Codex-Installation bietet. An der Datei `~/.codex/AGENTS.md` ändert ensemble-next in keinem Fall etwas.

4.4 So wählst du

Im Terminal ist die Arbeitsweise das dritte Argument:

```
# Standard: Claude Code allein, danach blinde Prüfung
./scripts/collab-launch.sh /pfad/zum/projekt "Aufgabe ..."

# Team: Codex leitet, Claude Code setzt um
./scripts/collab-launch.sh /pfad/zum/projekt "Aufgabe ..." codex,claude

# Team zu dritt, mit Rollenvorlage als viertem Argument
./scripts/collab-launch.sh /pfad/zum/projekt "Aufgabe ..." codex,claude,agy research
```

In Claude Code genügt der Befehl `/collab-next` mit der Aufgabe. Nennst du darin Agenten („mit Codex im Team“), gibt Claude sie weiter. Wer immer dieselbe Besetzung will, setzt einmal `COLLAB_AGENTS`. Es gilt: drittes Argument vor `COLLAB_AGENTS` vor dem Standard.

5 Welche Modelle arbeiten und wie du sie einstellst

ensemble-next bringt keine eigenen Modelle mit und legt auch keine fest. Es startet die Kommandozeilenprogramme `claude` und `codex` so, wie sie auf deinem Rechner eingerichtet sind. Welches Modell arbeitet, bestimmst du also dort, wo du es auch sonst bestimmst. Es gibt drei Stellen, eine je Rolle.

Rolle	Gestartet als	Modell kommt aus
Claude Code (setzt um)	`claude --permission-mode auto`	deiner Claude-Code-Einstellung: dem zuletzt mit `/model` gewählten Modell, dem Eintrag `model` in `~/.claude/settings.json` oder der Variable `ANTHROPIC_MODEL`
Codex im Team (leitet, plant)	`codex --dangerously-bypass-approvals-and-sandbox --no-daemon`	`~/.codex/config.toml`: Einträge `model` und `model_reasoning_effort`
Codex als Prüfer (urteilt)	`codex exec --sandbox read-only` mit Zusatzargumenten	denselben beiden Einträgen; ensemble-next liest sie aus und übergibt sie ausdrücklich

5.1 Claude Code auf ein Modell festlegen

Für einen Lauf oder dauerhaft für den Server genügt eine Variable beim Serverstart. Der Server reicht alle Variablen, die mit `ANTHROPIC_`, `OPENAI_` oder `ENSEMBLE_` beginnen, an die Agenten weiter:

```
ANTHROPIC_MODEL=opus npm run dev
```

Soll die Festlegung zu ensemble-next gehören und nicht zu deiner allgemeinen Einstellung, trägst du das Modell in die Startargumente ein. Die Datei `agents.json` beschreibt, wie jeder Agent gestartet wird:

```
"claude": {
  "command": "claude",
  "flags": ["--permission-mode", "auto", "--model", "opus", "--effort", "high"]
}
```

Am saubersten ist eine eigene Kopie dieser Datei außerhalb des Repositorys. Die Variable `ENSEMBLE_AGENTS_CONFIG` nennt beim Serverstart den Pfad zu dieser Kopie. So bleibt das Repository unverändert und ein Update überschreibt nichts.

5.2 Codex im Team festlegen

Ohne Zutun gilt, was in `~/.codex/config.toml` steht. Für eine Festlegung nur in ensemble-next kommen die Schalter ebenfalls in `agents.json`:

```
"codex": {
  "command": "codex",
  "flags": ["--dangerously-bypass-approvals-and-sandbox", "--no-daemon",
        "-m", "MODELLNAME", "-c", "model_reasoning_effort=\"high\""]
}
```

5.3 Den Prüfer festlegen

Der Prüfer nimmt dasselbe Modell und dieselbe Denkstufe wie dein normales Codex. Das ist Absicht: Ohne die persönliche Konfiguration fiele Codex sonst auf sein Standardmodell ohne Denkstufe zurück, und das Urteil wäre schwächer, ohne dass es jemand merkt. Soll der Prüfer ein anderes Modell bekommen als dein Arbeits-Codex, setzt du beim Serverstart:

```
COLLAB_REVIEW_MODEL=MODELLNAME COLLAB_REVIEW_EFFORT=high npm run dev
```

Die Datei `review/reviewer.json` hält für jede Prüfung fest, mit welchem Modell und welcher Denkstufe sie lief. Die Modellnamen sind die, die das jeweilige Programm selbst kennt; ensemble-next prüft sie nicht.

5.4 Was beim Einstellen zählt

- **Zwei Hersteller.** Der Wert der blinden Prüfung liegt darin, dass ein anderes Modell urteilt als das, das gearbeitet hat. Ein Prüfer aus demselben Haus übersieht eher dieselben Fehler.
- **Der Prüfer darf stark sein.** Eine Prüfung ist kurz. Ein gründliches Modell mit hoher Denkstufe kostet hier wenig und bringt viel.
- **Der Umsetzer trägt die Hauptlast.** Bei ihm entscheidet sich Verbrauch und Dauer. In der Messreihe entfielen im Standardweg rund 345 000 Tokens auf Claude und 25 000 auf Codex.
- **Variablen gelten ab Serverstart.** Der Server gibt sie an Agenten und Prüfer weiter. Eine Änderung wirkt erst nach einem Neustart des Servers.
- **Messwerte hängen am Modell.** Die Zahlen in Abschnitt 9 gelten für die dort genannten Modelle. Mit anderen Modellen können Dauer, Verbrauch und Qualität abweichen.

6 Die Sicherungen im Einzelnen

6.1 Abschlusscheck

Jedes Mal, wenn Claude Code stoppt, läuft ein Prüfskript. Es führt nur aus, was das Projekt selbst festlegt: die npm-Skripte `typecheck`, `lint`, `test` und `build`, soweit sie in `package.json` stehen, und `pytest`, sobald es Python-Tests gibt. Geraten wird nichts.

Ergebnis	Bedeutung
`PASS`	Alle Prüfungen sind grün.
`FAIL`	Mindestens eine Prüfung ist rot. Claude wird mit der Fehlerausgabe zurückgeschickt und soll die Ursache beheben, nicht die Prüfung abschwächen.
`INCOMPLETE`	Zeitüberschreitung, oder npm beziehungsweise pytest fehlt.
`SKIPPED`	Das Projekt legt keine Prüfungen fest. Der Prüfer kann „Tests grün“ dann nicht bestätigen.
`NO_CHANGES`	Es wurde nichts geändert.

Das Ergebnis steht in einem Protokoll, das der Prüfer zu sehen bekommt. Eine Behauptung „Tests grün“ ohne grünes Protokoll fällt damit auf.

6.2 Abbruchregeln für die Nacharbeit

Ein Agent, der immer wieder ändert und immer wieder scheitert, verbraucht Tokens ohne Ergebnis. Deshalb gelten Obergrenzen, gezählt ab der ersten roten Prüfung bis zur nächsten grünen:

Regel	Stan-dard	Variable
Versuche für denselben Änderungsstand	3	`ENSEMBLE_VERIFY_MAX_BLOCKS`
Nacharbeitsrunden insgesamt	6	`ENSEMBLE_VERIFY_MAX_ROUNDS`
Minuten seit der ersten roten Prüfung	30	`ENSEMBLE_VERIFY_MAX_MINUTES`
Änderungen in Folge mit unveränderter Fehlerausgabe	2	`ENSEMBLE_VERIFY_STALL_ROUNDS`

Der Wert `0` schaltet eine Regel ab. Ist eine Grenze erreicht, wird Claude nicht weiter zurückgeschickt. Der Stand bleibt als ungelöst markiert, das Protokoll nennt den Grund, und die Zusammenfassung zeigt ihn an. Die Standardwerte sind gesetzt, nicht gemessen.

6.3 Blinde Prüfung

Der Prüfer bekommt ein festes Bündel aus drei Teilen und sonst nichts: die Aufgabe im Wortlaut, den Diff seit dem Stand bei Arbeitsbeginn samt neuer Dateien, und das Protokoll des Abschlusschecks. Er sieht weder den Chat noch den Plan noch die Selbsteinschätzung des Umsetzers. Er läuft nur lesend in einem leeren Ordner und antwortet in einer festen Form:

Urteil	Bedeutung und nächster Schritt
`ACCEPTED`	Alle Kriterien erfüllt, Abschlusscheck grün, kein Blocker. Diff ansehen, dann selbst committen.
`CHANGES_REQUESTED`	Mindestens ein Kriterium nicht erfüllt oder nicht belegbar, die Lücken sind behebbar. Befunde lesen, nachbessern lassen oder selbst entscheiden.
`REJECTED`	Falsche Richtung, abgeschwächte Prüfungen oder Änderungen weit außerhalb der Aufgabe. Nicht übernehmen, Auftrag schärfen.

Nach einem Urteil wird nicht automatisch nachgearbeitet. Die Befunde gehen an dich.

6.4 Zitatprüfung

Der Prüfer muss jede Feststellung mit Datei und Zeile belegen. Ein Skript gleicht diese Fundstellen danach mit dem Diff und dem Projektordner ab. Es braucht dafür kein Modell und keine Tokens.

Status	Bedeutung
`grounded`	Mindestens eine Fundstelle liegt im geprüften Diff, keine ist falsch.
`ungrounded`	Keine Fundstelle liegt im Diff. Ein `ACCEPTED` gilt dann als nicht belegt.
`invalid`	Eine Fundstelle nennt eine Datei, die es nicht gibt, oder eine Zeile hinter dem Dateiende. Das Urteil gilt als nicht verlässlich.

Der Hinweis wird an das Urteil angehängt. Eine Fundstelle, die es gibt, beweist allerdings nicht, dass der Prüfer richtig geurteilt hat. Die Zitatprüfung fängt erfundene Belege ab, nicht falsche Schlüsse.

6.5 Bindung an die Prüfsumme

Zu jedem Urteil wird die Prüfsumme (SHA-256) des geprüften Stands festgehalten. Ändert sich der Projektordner, während die Prüfung läuft, steht das im Hinweis: Das Urteil gilt dann für den älteren Stand. Später, etwa vor einem Merge, lässt sich das nachprüfen:

```
./scripts/verdict-check.sh /tmp/ensemble-next/TEAM_ID/review
```

Rückgabe 0 heißt aktuell, 1 veraltet, 2 nicht feststellbar. Die Prüfsumme berechnet das Skript, nicht das Modell.

6.6 Abgeschotteter Prüfer

Der Prüfer startet ohne die persönliche Codex-Konfiguration: ohne MCP-Server, Plugins, Apps und Erinnerungen. Auf dem Entwicklungsrechner bekam er vorher 405 MCP-Werkzeuge mitgeliefert, jetzt keins. Zwei Prüfungen kosteten danach 9 900 und 10 400 Tokens statt 17 100 und 19 400, bei gleichen Urteilen. Lehnt eine neue Codex-Version die Zusatzargumente ab, läuft die Prüfung einmal auf dem alten Weg, statt zu scheitern. `COLLAB_REVIEW_ISOLATED=0` schaltet die Abschottung ab.

7 Einrichtung und erster Lauf

7.1 Voraussetzungen

- Linux mit Node.js, tmux, Python 3, Git und curl
- `claude` (Claude Code) und `codex`, beide installiert und angemeldet
- ein Projektordner, der ein Git-Repository mit mindestens einem Commit ist

7.2 Einrichten

```
git clone https://github.com/schrauberMK/ensemble-next.git
cd ensemble-next
npm install
npm run dev
```

Der Server läuft auf Port 23100 und bleibt in diesem Fenster offen. Ob er antwortet, zeigt:

```
curl -s localhost:23100/api/v1/health
```

In einem zweiten Terminal richtet ein Skript den Befehl `/collab-next` in Claude Code ein:

```
./scripts/setup-claude-code.sh
```

> Starte Server und tmux aus einem normalen Terminal, nicht aus einer Desktop-Anwendung heraus. Aus einer App gestartet erben die Agenten deren Umgebung, und Codex beendete sich in Tests dann ohne Meldung.

7.3 Erster Lauf

Ein Wegwerf-Projekt zum Ausprobieren:

```
mkdir -p ~/test-ensemble && cd ~/test-ensemble
git init -q && echo "# Test" > README.md && git add . && git commit -qm Start
```

Dann in Claude Code, im Projektordner:

```
/collab-next Schreibe eine Funktion ... mit Tests. Alle Tests muessen gruen sein.
```

Gute Aufträge nennen ein prüfbares Ziel: „mit Tests, alle Tests grün“, „`test/` nicht ändern“, „nur Datei X“. An diesen Kriterien misst der Prüfer das Ergebnis.

7.4 Ergebnis lesen

Alles liegt unter `/tmp/ensemble-next/TEAM_ID/` und verschwindet beim Neustart des Rechners.

Datei	Inhalt
`summary.txt`	Rückmeldung des Umsetzers, geänderte Dateien, Abschlusscheck
`review/verdict.txt`	Urteil mit Kriterien und Befunden, darunter der Hinweis der Zitatprüfung mit Prüfsumme
`review/verdict.raw.txt`	Urteil im Wortlaut des Prüfers, ohne Zusatz
`review/grounding.json`	Einzelheiten der Zitatprüfung und die Prüfsummen
`review/reviewer.json`	Modell, Denkstufe und ob abgeschottet geprüft wurde
`review/error.txt`	nur wenn die Prüfung kein Urteil liefern konnte

Die Prüfung ist fertig, sobald `review/.done` existiert.

8 Schalter

Alle Schalter sind Umgebungsvariablen. Die mit `ENSEMBLE_` und `COLLAB_REVIEW_` gelten ab Serverstart, die übrigen `COLLAB_`-Variablen für den einzelnen Aufruf.

Variable	Wirkung
`COLLAB_AGENTS`	Besetzung, etwa `codex,claude`; der erste Agent leitet
`COLLAB_TEMPLATE`	Rollenvorlage: `review`, `implement`, `research`, `debug`
`COLLAB_AUTO_REVIEW=0`	keine blinde Prüfung für diesen Start
`ENSEMBLE_AUTO_REVIEW=0`	keine blinde Prüfung für den ganzen Server
`COLLAB_REVIEW_MODEL`	Modell des Prüfers, abweichend von der Codex-Konfiguration
`COLLAB_REVIEW_EFFORT`	Denkstufe des Prüfers
`COLLAB_REVIEW_ISOLATED=0`	Prüfer mit voller Codex-Konfiguration starten
`COLLAB_REVIEW_RETRY_S`	Pause vor einem neuen Versuch, wenn Codex überlastet ist (120 s, höchstens 3 Versuche)
`COLLAB_AGENT_WAIT_SECS`	Wartezeit auf die erste Nachricht beim Start (60 s)
`ENSEMBLE_VERIFY_ON_STOP=0`	Abschlusscheck abschalten
`ENSEMBLE_VERIFY_TIMEOUT_MS`	Zeitgrenze je Prüfung im Abschlusscheck (120 000)
`ENSEMBLE_VERIFY_MAX_BLOCKS`	Versuche je Änderungsstand (3)
`ENSEMBLE_VERIFY_MAX_ROUNDS`	Nacharbeitsrunden insgesamt (6)
`ENSEMBLE_VERIFY_MAX_MINUTES`	Minuten Nacharbeit (30)
`ENSEMBLE_VERIFY_STALL_ROUNDS`	Änderungen in Folge ohne Wirkung (2)
`ENSEMBLE_AGENTS_CONFIG`	eigene `agents.json` statt der im Repository
`ANTHROPIC_MODEL`	Modell für Claude Code, wird an den Agenten weitergereicht

9 Messwerte

Bevor der Standardweg festgelegt wurde, liefen beide Arbeitsweisen gegeneinander: vier kleine Aufgaben (Fehlerbehebung, Funktion über mehrere Dateien, Umbau mit Auflage, absichtlich widersprüchlicher Auftrag), je zweimal je Arbeitsweise, zusammen 16 Läufe am 30.10.2026. Die Entscheidungsregel stand vorher fest: Der Standardweg gewinnt nur, wenn die Qualität mindestens gleich und Dauer oder Verbrauch um mindestens 20 Prozent besser sind.

Kennzahl	Team (Codex plant)	Standard (Claude allein)
Läufe	8	8
versteckte Abnahmetests bestanden	8	8
Verstöße gegen den Schreibbereich	0	0
blind akzeptiert	6	6
Widerspruch richtig gemeldet	2 von 2	2 von 2
Dauer, Median	156 s	57 s
Claude-Tokens, Median	607 000	345 000
Codex-Tokens, Median	313 000	25 000
Zurückweisungen durch den Abschlusscheck	6	0

Gemessen wurde mit Claude Opus 5.5 und, auf der Codex-Seite, GPT-6.1 Sol. Die je zwei nicht akzeptierten Läufe gehören zur widersprüchlichen Aufgabe; dort ist `CHANGES_REQUESTED` das richtige Urteil.

Das ist ein Pilotversuch mit wenigen, kleinen Aufgaben auf einem Rechner. Er zeigt eine Tendenz für diese Art von Aufgaben und ist kein allgemeiner Beweis. Bei großen oder offenen Aufgaben kann das Team im Vorteil sein. Die Rohdaten liegen unter `bench/results/`, die Werkzeuge unter `bench/`; die Messung lässt sich mit eigenen Aufgaben wiederholen.

10 Grenzen und bekannte Punkte

- **Die blinde Prüfung ist kein Ersatz für eigenes Nachsehen.** Sie senkt die Zahl der Fälle, in denen etwas Falsches als fertig gilt. Ein `ACCEPTED` heißt: Die genannten Kriterien sind nach Ansicht eines zweiten Modells erfüllt.
- **Ohne Tests kein Beleg.** Legt das Projekt keine Prüfungen fest, kann niemand „Tests grün“ bestätigen.
- **Die globale Codex-Anweisungsdatei wird weiter geladen.** Auch der abgeschottete Prüfer liest `~/.codex/AGENTS.md`. Ein Schalter dagegen zeigte in codex-cli 0.167.1 keine Wirkung.
- **Zeilenangaben werden nur grob geprüft.** Die Zitatprüfung vergleicht mit Bereichen, weil ein Prüfer beim Zählen im Diff leicht ein paar Zeilen danebenliegt.
- **„Kein Fortschritt“ erkennt nur wortgleiche Fehlerausgaben.** Ändert sich die Ausgabe, gilt das als Fortschritt, auch wenn es keiner ist. Dann greifen Runden- und Zeitgrenze.
- **Updates der beiden Programme können den Start stören.** Das Erkennen von Startdialogen hängt an Zeichen im Terminalfenster und musste schon nachgezogen werden.
- **Aus einer Desktop-App gestartet** kann Codex im Team ausfallen. Server und tmux gehören in ein normales Terminal.
- **Ein Rechner, ein Nutzer.** Der Server hat keine Zugangskontrolle und lauscht lokal. Er ist nicht für den Betrieb im Netz gedacht.
- **Testläufe hinterlassen Reste.** Die Testsuite lässt gelegentlich einen Testserver zurück, und Codex merkt sich freigegebene Ordner in seiner Konfiguration.

Das Entwicklungstagebuch `NEXT.md` im Repository führt alle Befunde, Fehlversuche und offenen Punkte mit Datum.

11 Lizenz und Haftung

ensemble-next steht unter der **MIT-Lizenz**, wie das Ursprungsprojekt. Du darfst die Software nutzen, ändern und weitergeben, auch gewerblich, solange der Urheberrechtsvermerk und der Lizenztext erhalten bleiben. Maßgeblich ist die Datei `LICENSE` im Repository.

Die Software wird ohne jede Gewährleistung bereitgestellt, weder ausdrücklich noch stillschweigend. Das schließt die Eignung für einen bestimmten Zweck ein. Die Urheber haften nicht für Schäden, die aus der Nutzung entstehen, etwa für verlorene Daten, fehlerhaften Code oder Kosten bei den Anbietern der KI-Modelle.

Die verwandten Projekte haben eigene Lizenzen. CC_GodMode und sein Gegenstück für Codex sind proprietär und nicht Teil dieses Repositorys. „Claude“ und „Claude Code“ sind Marken von Anthropic, „Codex“ und „ChatGPT“ Marken von OpenAI. ensemble-next ist ein unabhängiges Projekt und wird von keinem der beiden Unternehmen unterstützt oder geprüft.